Структура пояснительной записки к курсовой работе

Структура пояснительной записки к курсовой работе
«Разработка веб-приложения»

Объем: 20–30 стр. (без приложений)

ТИТУЛЬНЫЙ ЛИСТ
(по Стандарту Предприятия)

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ
(по шаблону)

СОДЕРЖАНИЕ
(Автоматически генерируется в Word)

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СОКРАЩЕНИЙ И ТЕРМИНОВ (опционально, 1 стр.)

- API – Application Programming Interface, программный интерфейс приложения.
- SPA – Single Page Application, одностраничное приложение.
- JSX – JavaScript XML, синтаксическое расширение для JavaScript.
- REST – Representational State Transfer, стиль архитектуры веб-сервисов.
- JWT – JSON Web Token, токен для аутентификации.
- ORM – Object-Relational Mapping, объектно-реляционное отображение.
- CI/CD – Continuous Integration / Continuous Deployment, непрерывная интеграция и развертывание.
- Деплой – Развертывание приложения на сервере.

ВВЕДЕНИЕ (1–1.5 стр.)

- **Актуальность темы:** Роль веб-приложений в современном цифровом мире, важность выбранной предметной области (напр., управления задачами,

электронной коммерции).

- **Цель работы:** Разработать полнофункциональное веб-приложение «[Название]» на современном стеке технологий.
- **Задачи работы:**
    1. Провести анализ предметной области и сформулировать требования.
    2. Спроектировать архитектуру и выбрать технологический стек.
    3. Реализовать серверную часть (REST API, БД, аутентификацию).
    4. Разработать клиентскую часть (React SPA с адаптивным интерфейсом).
    5. Проинтегрировать части, провести тестирование и развернуть приложение.
- **Объект и предмет исследования:** Объект – процесс разработки веб-приложений. Предмет – методы и технологии создания SPA с использованием React, Node.js и PostgreSQL/MongoDB.
- **Практическая значимость:** Приложение может быть использовано для [конкретная цель, напр., организации учебного процесса, малого бизнеса].
- **Структура работы:** «Пояснительная записка состоит из введения, трех глав, заключения, списка использованных источников и приложений».

ГЛАВА 1. АНАЛИЗ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ (7–8 стр.)

- **1.1. Анализ предметной области и формулировка требований** (2 стр.)
    - Описание решаемой проблемы.
    - Список функциональных требований (FR) в виде пользовательских историй: «Пользователь может...».
    - Ключевые нефункциональные требования (NFR): производительность, безопасность, адаптивность.
- **1.2. Проектирование архитектуры приложения** (3 стр.)
    - Логическая архитектура: Диаграмма вариантов использования (Use Case Diagram) для основных акторов (Пользователь, Администратор).
    - Физическая архитектура: Схема «Клиент-Сервер-БД» с указанием технологий (React, Express, Nginx, PostgreSQL). Альтернатива: Контекстная

диаграмма C4 (Уровень 1).

    - Проектирование REST API: Таблица основных эндпоинтов (Метод, URL, Описание, Пример тела запроса/ответа).

- **1.3. Проектирование структуры данных** (2 стр.)
    - ER-диаграмма базы данных (4–6 сущностей, ключевые атрибуты и связи).
    - Краткое описание таблиц/коллекций.

ГЛАВА 2. РЕАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ (12–13 стр.)

- **2.1. Выбор и обоснование технологического стека** (1.5 стр.)
    - Таблица сравнения и обоснование выбора:
        - **Frontend:** React (компонентный подход, Virtual DOM, богатая экосистема).
        - **Backend:** Node.js + Express.js (асинхронность, единый язык JavaScript, скорость разработки).
        - **База данных:** PostgreSQL (надежность, транзакции, SQL) или MongoDB (гибкость схемы, документная модель).
        - **Дополнительно:** Git (контроль версий), Docker (контейнеризация), Postman (тестирование API), VS Code (редактор кода).
- **2.2. Реализация серверной части (Backend)** (4 стр.)
    - **2.2.1. Структура проекта и настройка:** Описание папок (models, routes, controllers, middleware), файла package.json.
    - **2.2.2. Модели данных и работа с БД:** Код ключевой модели с использованием Sequelize/Mongoose с комментариями.
    - **2.2.3. Реализация REST API:** Пример кода контроллера с обработкой CRUD-операций, валидацией.
    - **2.2.4. Аутентификация и авторизация:** Схема работы JWT. Пример кода middleware для проверки токена и защищенного маршрута.
    - **2.2.5. Обработка ошибок и безопасность:** Использование helmet, cors, валидация входящих данных, хранение паролей (bcrypt).

- **2.3. Реализация клиентской части (Frontend)** (4 стр.)
    - **2.3.1. Структура React-приложения:** Описание папок (src/components, src/pages, src/context).
    - **2.3.2. Маршрутизация:** Настройка React Router, пример объявления защищенных и публичных маршрутов.
    - **2.3.3. Ключевые компоненты и состояние:** Пример компонента с использованием хуков useState, useEffect. Описание глобального состояния (Context API или useReducer).
    - **2.3.4. Взаимодействие с сервером:** Пример использования fetch или axios для запроса к API, обработка состояний загрузки и ошибок.
    - **2.3.5. Адаптивный интерфейс:** Подход к стилизации (CSS-модули, Styled Components), применение Flexbox/Grid.
- **2.4. Интеграция, сборка и развертывание** (2.5 стр.)
    - **2.4.1. Интеграция frontend и backend:** Настройка прокси в режиме разработки, работа с CORS.
    - **2.4.2. Контейнеризация с Docker:** Приведение Dockerfile для backend и (опционально) frontend. Описание файла docker-compose.yml для запуска всего стека.
    - **2.4.3. Деплой:** Краткое описание процесса развертывания на облачной платформе (напр., Render для backend и БД, Netlify для frontend).
    - **2.4.4. Тестирование:** Описание написанных модульных тестов (Jest для API) или ручных тестовых сценариев.

ГЛАВА 3. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ И ТЕСТИРОВАНИЕ (4–5 стр.)

- **3.1. Руководство пользователя** (2.5 стр.)
    - Требования к окружению: Требуемая версия браузера.
    - Инструкция по запуску (для разработчиков): Команды git clone, npm i, docker-compose up.
    - Основные сценарии использования: Пошаговое описание 2–3 ключевых функций (напр., «Регистрация нового пользователя», «Создание новой задачи»).

    - Скриншоты интерфейса: 3–4 ключевых скриншота с краткими пояснениями (Главная страница, Форма входа, Панель управления).

- **3.2. Контрольные примеры и тестирование** (1.5 стр.)
    - Таблица с контрольными примерами: Исходные данные → Действия пользователя → Ожидаемый результат → Фактический результат (Пройдено/Не пройдено).
    - Вывод о соответствии приложения заданным требованиям.

ЗАКЛЮЧЕНИЕ (1 стр.)

- Краткий перечень выполненных задач: «В ходе работы были решены все поставленные задачи: проанализированы требования, спроектирована архитектура...».
- Констатация достижения цели: «Цель работы – разработка веб-приложения «[Название]» – достигнута».
- Краткое описание результата: «В результате создано рабочее веб-приложение, доступное по адресу [URL]. Приложение реализует функции [перечень основных]».
- Перспективы развития: «В дальнейшем возможно добавление [напр., PUSH-уведомлений, мобильного приложения, аналитической панели]».

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (1 стр.)
(15–20 источников по ГОСТу или Стандарту Предприятия)

- Официальная документация React, Node.js, Express.
- Учебники из списка учебной программы.
- Книги (не старше 5 лет).
- Статьи с Habr, MDN Web Docs, Stack Overflow (актуальные, 2022–2026 гг.).
- Репозитории GitHub с примерами.

ПРИЛОЖЕНИЯ *(Не входят в общий объем 25–30 стр.)*

- **ПРИЛОЖЕНИЕ А –** Полный листинг кода (если требуется).
- **ПРИЛОЖЕНИЕ Б –** Исходные данные для тестирования.

- **ПРИЛОЖЕНИЕ В –** Полные скриншоты всех экранов приложения.
- **ПРИЛОЖЕНИЕ Г –** Рекомендации по выбору бесплатных сервисов для хостинга, баз данных и CI/CD.

Приложение Г: Рекомендации по выбору бесплатных сервисов

В связи с текущей геополитической ситуацией и санкционными ограничениями, традиционные облачные сервисы (например, Heroku, Vercel) могут быть недоступны для пользователей из Беларуси. В курсовой работе рекомендуется использовать следующий стек полностью бесплатных и альтернативных сервисов:

1. Хостинг (Деплой)

- **Frontend (React): Netlify** или **GitHub Pages**. Эти сервисы предоставляют простой и быстрый деплой статических приложений и, как правило, остаются доступными. Поддерживают бесплатные SSL-сертификаты и кастомные домены.
- **Backend (Node.js) и База данных: Render** или **Supabase**.
    - **Render** (ранее известный как OnRender) предоставляет бесплатный тариф для веб-сервисов (750 часов в месяц) и managed базы данных PostgreSQL. Он широко известен как одна из лучших бесплатных альтернатив Heroku.
    - **Supabase** — это open-source альтернатива Firebase. Предоставляет бесплатно managed PostgreSQL базу данных, аутентификацию, API и реальное время. Это отличный выбор, если не хочется настраивать отдельный бэкенд.
- **Self-Hosted (Собственный сервер):** Если требуется полный контроль, можно рассмотреть open-source PaaS платформы, такие как **Coolify**, которые можно развернуть на собственном VPS сервере. Это решение для тех, кто хочет развернуть у себя аналог Heroku.

2. Базы данных (Database as a Service)

- **PostgreSQL: Supabase** (предоставляет полноценную БД), **Neon** (Serverless PostgreSQL с бесплатным тарифом, включающим 500 MB хранилища), или **Render** (как упоминалось выше).
- **MongoDB: MongoDB Atlas**. Это облачный сервис самой MongoDB, который имеет щедрый бесплатный тариф (512 MB хранилища).

3. CI/CD (Непрерывная интеграция и развертывание)

- **GitHub Actions:** Лучший выбор для репозиториев на GitHub. Для публичных репозиториев предоставляется неограниченное время сборки, а для приватных — 2000 минут в месяц. Полностью интегрирован с вашим кодом и позволяет настроить автоматический деплой на сервера (Render, Netlify и др.) при каждом пуше.

4. Мониторинг ошибок (Error Tracking)

- **GlitchTip:** Это open-source и бесплатная альтернатива Sentry, которую можно использовать как в облаке, так и развернуть на своем сервере. GlitchTip полностью совместима с клиентским SDK Sentry, поэтому интеграция в код не потребует его замены.

5. Веб-Аналитика

- **Альтернативы Google Analytics:** В условиях санкций можно использовать **Яндекс.Метрику** — бесплатный и мощный сервис. Если важна приватность и открытость, можно рассмотреть open-source решения: **Rybbit** или **Plausible**, которые можно установить на свой сервер.